Deep Learning: From Multilayer Perceptrons to Transformers

Syllabus

Rishov Nag Deepanjan Mitra

Contents

Deep Learning: From Multilayer Perceptrons to Transformers

Graduate course · 14 weeks

Companion documents: `readings.pdf` (annotated bibliography), `technical-notes.pdf` (derivations and supplementary material).

---

1. Course description

This course develops deep learning from the loss-minimization frame through to the Transformer architecture and its modern descendants. It is organized around a single analytical question: **what structure does an architecture assume about its data, and what does that assumption buy?** MLPs, convolutional networks, recurrent networks, and Transformers are treated as points in a space of inductive biases rather than as a historical succession.

The first half of the course builds the Transformer from the problems that motivated it — the fixed-context bottleneck in sequence-to-sequence models is something students measure in a lab before attention is introduced in lecture. The second half is devoted to what the original paper does not explain: why these models generalize, how their hyperparameters transfer across scale, what pretraining does, and how to evaluate any of it.

Implementation is continuous. Students build a reverse-mode autodiff engine in NumPy, extend it through convolution and recurrence, and then implement a Transformer using only tensor primitives — no library layer that the assignment asks them to write.

Learning outcomes

On completion, a student should be able to:

1. Derive backpropagation as reverse-mode automatic differentiation on an arbitrary computational graph, and implement it without a framework.
2. State precisely what universal approximation theorems guarantee, where Barron's theorem improves on them, and why depth provably helps for some function classes.
3. Diagnose training pathologies — vanishing and exploding gradients, dead units, loss spikes, divergence under learning-rate increase — from variance propagation and Jacobian arguments rather than by heuristic.

4. Analyze any architecture in terms of weight sharing, equivariance, effective receptive field, and maximum path length between dependent positions.
5. Reconstruct the Transformer from its motivating constraints and justify every component — the $1/\sqrt{d_k}$ scaling, residual placement, normalization position, positional encoding — from first principles.
6. Explain why classical capacity control fails for overparameterized models, and articulate the current state of the disagreement about what replaces it.
7. Predict how learning rates and initialization scales must change with model width, and why naive scaling fails.
8. Read a contemporary architecture or scaling paper critically: identify which ablation actually carries the central claim.

Explicit scope limits

This course does **not** cover generative modeling in depth. Variational autoencoders, GANs, normalizing flows, and diffusion models receive one lecture of context and no assignments. This is the largest deliberate omission and it is stated in Week 1 so students can plan. Reinforcement learning appears only as it bears on post-training. Graph neural networks appear only in the unification lecture. Students wanting these should take the dedicated courses.

---

2. Prerequisites

- **Probability and statistics** at the level of a rigorous undergraduate course. Gaussians, expectation, variance, conditional independence, maximum likelihood.
- **Linear algebra.** Eigendecomposition, SVD, spectral norm, positive-definiteness, matrix calculus. Students should be able to differentiate a scalar with respect to a matrix without looking it up.
- **Multivariable calculus.** Chain rule on compositions of vector-valued functions.
- **One prior machine learning course.** Formulating cost functions, taking derivatives, optimizing by gradient descent.
- **Python proficiency.** All assignments use NumPy and PyTorch. Assignments provide minimal scaffolding; the code volume is substantially higher than in a typical ML course.

Lab 1 (Week 1) covers NumPy and the matrix-calculus handout, which is examinable. Lab 2 (Week 2) covers PyTorch.

---

3. Texts

Primary

- Prince, *Understanding Deep Learning* (MIT Press, 2023). Free online. The main text.
- Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016). Free online. Reference for Parts I–II.

Secondary, assigned by section

- Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. draft — Chapters 2, 9, 10 for tokenization, Transformers, masked language models.
- Torralba, Isola & Freeman, *Foundations of Computer Vision* (MIT Press, 2024). Free online. Convolution and representation-learning chapters.
- Bishop & Bishop, *Deep Learning: Foundations and Concepts* (Springer, 2024). Alternative treatment for students who prefer a probabilistic framing.

Theory supplements

- Telgarsky, *Deep Learning Theory* lecture notes — §2 and §5 for approximation.
- Roberts, Yaida & Hanin, *The Principles of Deep Learning Theory* (CUP, 2022) — for students who want the effective-theory treatment of initialization.

Full annotated bibliography in `readings.pdf`.

---

4. Assessment

Component	Weight
Weekly laboratories (14 set, best 12 count)	20%
Problem sets (5 × 5%)	25%
Midterm exam	20%
Final project	30%
Participation	5%

Grading philosophy. Every result derived on the board is *measured* by you before the term ends — not illustrated, measured. The laboratories are where that happens. Problem sets and the exam test whether you can derive things; the laboratories test whether you can make a derivation predict a number and then check it; the project tests whether you can investigate something. Nothing here tests whether you can recall an architecture diagram.

Roughly 57% of the total mark is implementation and measurement and 43% is written argument. Without implementation the course does not mean anything; without the midterm there is no instrument that tests whether you can reproduce an argument away from a machine.

Laboratories

One 50-minute laboratory per week, in the GPU lab, Friday. Fourteen are set and **the best twelve count** — the two dropped scores exist so that a missed session needs no paperwork. Each is worth 1.67% and has three parts, always in this order:

(a) Make it run — a scaffolded implementation with unit tests. Auto-graded. 50% of the lab mark. **(b) Measure something** — a specified plot or number, produced by your own code. 30%. **(c) One question** — under 150 words, answerable only by someone who has looked at their own output from (b). 20%.

Part (c) is the part that matters, and it is a question about *your* numbers. Full briefs are in `assignments.pdf`.

Several laboratories build on earlier ones — $5 \to \{6 \to 7 \to 14,\ 13\}$, and $9 \to 14$ — so a later lab's starter repository ships a `--use-reference` flag supplying a working implementation of the prior one. That reference is distributed as an installed package, not as readable source, precisely so it can go out on time without disclosing a solution still inside another student's late-day window. The same flag covers PS2 B1 (from Lab 4) and PS4 B1 (from Lab 9). Falling behind on one laboratory never blocks the next.

Problem sets

Five sets, **5% each**. Each has a **Part A** (derivations, 35%) and a **Part B** (implementation, 65%), and the two are linked: at least one Part A result is verified numerically by Part B code. **A numerical mismatch that you do not flag loses more marks than a wrong derivation that you do.** Derivations marked (E) are on the exam-eligible list in `technical-notes.pdf` §22.

Late policy

Ten penalty-free late days are granted automatically. Beyond that, a submission n days late is multiplied by $(1 - n/14)$; nothing is accepted more than 7 days late. Partial days round up. Late days cover the ordinary contingencies of life — being behind, a conference, a bad week — and no further extensions are granted for these. Genuine medical or personal emergencies go through the relevant student support office.

Two clarifications, because the sentences above have been read both ways. The ten days are a **single term-wide pool**, not ten days per assignment; and the seven-day cap is **per assignment**, so the $(1 - n/14)$ multiplier applies once the pool is exhausted. Non-submission of a laboratory is not a late-policy matter at all — that is what the two dropped scores are for.

Collaboration

Discussion with peers, TAs, and instructors is encouraged. Write-ups must be individual and must reflect your own work. Do not copy or share complete solutions, and do not ask others to confirm whether an answer is correct. List collaborators at the top of each submission.

Laboratories are the one exception, and in the permissive direction: collaboration during the session is *open*, because the auto-graded tests make copying pointless and part (c) is about your own measurements. Submission is still individual.

AI assistants

The policy for AI assistants is identical to the policy for human assistants. You may ask an AI anything you could ask a TA in office hours: clarifications, background, debugging help, tips for getting started. You may not ask it to do the assignment, exactly as you may not ask an expert friend to do it. If it is ever unclear, imagine the assistant is a person and apply the same norm.

You *should* use these systems — this is a deep learning course and learning where they succeed and fail is part of the content. Declare what you used and how in `AI_USE.md` at the root of each

submission. **Undeclared use is an integrity violation; declared use is not penalized.**

Two carve-outs, where assistance of any kind is not permitted. **Part A derivations** — the derivation is the assessment. And **Lab 5, part (c)** — the entire pedagogical mechanism of Week 5 depends on you not being told the answer, and that laboratory cannot be run twice.

Compute

Assignments run on the departmental GPU cluster, on a course partition with per-student fair share. Every brief states a **total** GPU-hour budget rather than a per-run figure, because the per-run figure is the one that misleads:

Lab	1	2	3	4	5	6	7
GPU-h	0	0.2	0.2	0.5	2.5	1.5	0.5

Lab	8	9	10	11	12	13	14
GPU-h	2	6	4	1	1	0.5	1

Set	PS1	PS2	PS3	PS4	PS5
GPU-h	0.5	10	6	12	15

That is about 65 GPU-hours per student across the term, concentrated in Weeks 5–11, with projects scoped to a further 20. Three items — Lab 9, Lab 11 and PS4 — run against **reserved cluster windows** announced in Week 1 and pinned in the course calendar; students who leave those to the last day will not get the GPUs. Being compute-constrained is the normal research condition and creativity under it is part of what is being assessed.

Submission

Everything is a push to your fork of the assignment repository: source at the paths the test suite expects, `figures/` as PDF, `REPORT.md` (laboratories) or `report.pdf` (problem sets), and `AI_USE.md`. Nothing else is read. Public tests are visible and runnable from day one; private tests run on submission and probe degenerate shapes, fully-masked rows, batch size 1, and non-contiguous tensors. `run.sh` is the single entry point the autograder calls, and it never invokes Jupyter — your code must work headless.

---

5. Weekly schedule

Two 80-minute lectures per week plus a 50-minute laboratory.

Wk	Theme	Laboratory	Due / out
1	The loss-minimization frame	L1 Numerics before networks	**PS1 out**
2	Backpropagation and AD	L2 The zoo of silent failures	—
3	Approximation theory, initialization	L3 The exponential-failure table	PS1 due
4	Grids: convolutional networks	L4 Receptive fields, measured	PS2 out
5	Memory: recurrence and its limits	L5 **The bottleneck**	—
6	Attention	L6 Attention, and the payoff	PS2 due, PS3 out
7	The Transformer	L7 The parameter count	Proposal due
8	Generalization · Midterm	L8 Memorization	—
9	Optimization at scale	L9 Coordinate check, roofline	**PS3 due**, PS4 out
10	Representation learning	L10 Collapse, and preventing it	—
11	Pretraining and scaling	L11 Scaling laws on a budget	PS4 due, PS5 out, milestone
12	Post-training and adaptation	L12 LoRA and RoPE	—
13	The practitioner's week	L13 The metric is an instrument	PS5 due
14	Inference, retrospect	L14 Bandwidth, and cheating it	Project due, poster

Two dates differ from earlier editions of this syllabus. **PS1 now goes out in Week 1**, not Week 2 — it was the only one-week set and the one built from nothing. **PS3 is now due in Week 9**, not Week 8, so that a four-page report no longer lands in the same week as the midterm.

PART I — FOUNDATIONS

Week 1 · The loss-minimization frame

L1 — Setup and linear models. Course overview and scope limits. Supervised learning and empirical risk minimization. The bias–variance decomposition and a warning that it will not survive Week 8. Loss functions as negative log-likelihoods: squared error corresponds to a Gaussian observation model, cross-entropy to a categorical one. Linear classifiers viewed

algebraically, visually as template matching, and geometrically as hyperplanes. Softmax and its numerical stabilization. The XOR obstruction.

Objectives: derive cross-entropy from a categorical likelihood; explain why softmax requires the max-subtraction trick; state what a linear model cannot represent.

L2 — Regularization and optimization. Taught deliberately *before* backpropagation, following CS231n: students should see the optimization problem before the mechanics of gradient computation. Gradient descent and its stochastic approximation. Minibatch noise. Momentum and the heavy-ball view; Nesterov's correction. AdaGrad, RMSProp, Adam. Learning-rate schedules: step, cosine, linear warmup. Weight decay, its equivalence to L_2 under plain SGD, and the non-equivalence under Adam that motivates AdamW.

Objectives: explain when momentum helps and when it destabilizes; state the AdamW correction and why it matters; read a divergent loss curve.

Readings: Prince Ch. 2–6; Kingma & Ba (2015); Loshchilov & Hutter (2019); Goh, *Why Momentum Really Works*. **Lab 1: Numerics before networks.** NumPy review and the matrix-calculus handout; stable softmax and log-sum-exp; the float32 overflow boundary; iterations-to-tolerance against condition number for gradient descent and heavy ball. **PS1 out.**

Week 2 · Backpropagation and automatic differentiation

L1 — The chain rule on computational graphs. Forward-mode versus reverse-mode AD. Why reverse mode costs $O(1)$ forward passes for a scalar output, and why that decides the question for loss minimization. Vector–Jacobian products as the primitive operation. Deriving VJPs for the layers we will need: affine, elementwise nonlinearity, softmax, cross-entropy, and the composition rules that assemble them.

L2 — Differentiable programming and practice. Rumelhart, Hinton & Williams in their original framing, and what was and was not new in 1986. Memory–compute tradeoffs; activation checkpointing. Numerical gradient checking as a debugging discipline, including the choice of ϵ and the relative-error criterion. Common failure modes: broadcasting errors that silently produce wrong gradients, in-place operations, detached tensors.

Objectives: implement a VJP for an arbitrary layer; explain the memory cost of the backward pass; gradient-check a hand-written layer to 10^{-7} relative error.

Readings: Rumelhart, Hinton & Williams (1986); Baydin et al. (2018); Karpathy, *Yes you should understand backprop*; `technical-notes.pdf` §2–3. **Lab 2: The zoo of silent failures** — also the PyTorch introduction. Six training scripts, each broken in exactly one way, none of which raises. Diagnose and repair.

Week 3 · Approximation theory and initialization

L1 — What can a network represent? Universal approximation: Cybenko (1989) and Hornik (1991), read carefully for what is actually proved — density in $C(K)$ on compacta, with no bound on width and no claim about learnability. Barron's theorem and the dimension-independent

$O(1/\sqrt{m})$ rate for functions with integrable Fourier moment; why this is the more useful result. Depth separation: Telgarsky's sawtooth construction showing functions representable by a deep network that require exponential width at shallow depth. Eldan & Shamir's radial-function separation.

This lecture is the single largest gap in most deep learning curricula. It is what prevents "universal approximation" from being cited as if it explained anything.

L2 — Signal propagation and initialization. Variance of activations and gradients through depth. Deriving Xavier/Glorot initialization from the requirement that forward and backward variance be preserved, and the He correction for ReLU's factor of two. Orthogonal initialization and the dynamical-isometry picture. Dead ReLUs. What actually goes wrong at bad initialization scales, demonstrated live.

Objectives: state universal approximation precisely and say what it does not give you; derive the He initialization constant; predict the activation variance at layer 50 given an initialization scheme.

Readings: Telgarsky notes §2, §5; Barron (1993); Glorot & Bengio (2010); He et al. (2015); `technical-notes.pdf` §4–5. **Lab 3: Regenerating the exponential-failure table. PS1 due.**

PART II — ARCHITECTURES AS INDUCTIVE BIAS

Week 4 · Grids: convolutional networks

L1 — Convolution as constrained linear algebra. The convolutional layer derived as an affine layer with weight sharing and locality imposed. Translation equivariance and where it breaks (boundaries, stride, pooling). Stride, padding, dilation. Receptive-field arithmetic and effective versus theoretical receptive field. Pooling and the equivariance-to-invariance trade. Implementation via `im2col`, and why that framing makes the FLOP accounting obvious.

L2 — Architecture lineage and normalization. LeNet-5 → AlexNet → VGG → ResNet, read as a sequence of answers to specific failures. Residual connections: identity paths, gradient highways, the loss-landscape smoothing evidence, and pre-activation ordering. Batch normalization: Ioffe & Szegedy's internal-covariate-shift story presented alongside Santurkar et al.'s refutation, so students see a canonical case of a technique that works for reasons other than those originally given. LayerNorm, RMSNorm, GroupNorm, and why sequence models cannot use batch statistics.

Both residual connections and layer normalization are flagged as prerequisites for Week 7. The Transformer inherits them wholesale.

Objectives: compute the receptive field and parameter count of a given CNN; explain why ResNets train at depth 100 and VGGs do not; state what BatchNorm does at test time and why that differs from training.

Readings: Prince Ch. 10–11; LeCun et al. (1998) §I–II; Krizhevsky et al. (2012); He et al. (2016a, 2016b); Ioffe & Szegedy (2015); Ba et al. (2016); Santurkar et al. (2018); `technical-notes.pdf`

§6–7. **Lab 4: Receptive fields, measured not counted. PS2 out.**

Week 5 · Memory: recurrence and its limits

L1 — Recurrent networks. Sequence modeling and the parameter-sharing-across-time argument. Elman networks. Backpropagation through time and its truncation. The vanishing/exploding gradient problem derived properly: the gradient across $t - k$ steps is a product of Jacobians, bounded by $(\gamma\lambda_{\max})^{t-k}$; exponential decay or blowup is the generic case. Gradient clipping as a principled response to the exploding half only. LSTM and the constant error carousel — the additive cell-state path is the same idea as a residual connection, arriving eleven years earlier. GRU. Greff et al.'s ablation of which gates matter.

L2 — Language modeling and the bottleneck. Neural language models from Bengio et al. (2003). Distributed representations: word2vec, GloVe, and Levy & Goldberg's result that skip-gram with negative sampling implicitly factorizes a shifted PMI matrix. Subword tokenization with BPE. Encoder–decoder sequence-to-sequence models; teacher forcing and exposure bias; beam search; BLEU and what it fails to measure.

Then the central problem of the course. A single fixed-dimensional context vector must summarize a source sentence of arbitrary length. Cho et al.'s length-degradation curves. Sutskever's source-reversal trick, presented as a symptom rather than a cure.

Objectives: derive the vanishing-gradient bound; explain why LSTM's cell path avoids it; state the information-theoretic problem with a fixed context vector.

Readings: Bengio et al. (1994); Hochreiter & Schmidhuber (1997); Pascanu et al. (2013); Bengio et al. (2003); Sennrich et al. (2016); Sutskever et al. (2014); Cho et al. (2014) §4; Olah, *Understanding LSTM Networks*; `technical-notes.pdf` §8–9.

Lab 5 — the pivotal laboratory. Train a small seq2seq translator. **Plot BLEU against source-sentence length, at three context dimensions, and observe that quadrupling the dimension does not flatten the curve.** Then propose, in your own words, the minimal architectural change that removes the cause. No one is told the word "attention" until Monday, and the word appears nowhere in the laboratory, its repository, or its tests. The motivation for the next two weeks must be something students measured, not something they were told.

Wednesday's lecture will already have named the cause — a fixed-capacity summary — so part (c) is not marked for restating it, and certainly not for naming the published mechanism. It is marked on whether the proposed change actually removes the constraint rather than enlarging it, and on whether the student can point to the feature of their own three curves that rules out a training deficiency.

Week 6 · Attention

L1 — Attention as alignment. Bahdanau, Cho & Bengio (2015): a distinct context vector computed at each decoder step as a learned, differentiable soft alignment over encoder states. Derived directly as the fix for Friday's measurement. Luong et al. (2015): multiplicative and

dot-product scoring, global versus local attention, input-feeding. Attention outside translation — Xu et al. (2015), soft versus hard attention, and the REINFORCE estimator the hard variant requires. Attention maps as interpretability evidence, and the Jain & Wallace / Wiegreffe & Pinter exchange on whether that inference is valid.

L2 — Self-attention. Dropping the encoder–decoder distinction: queries, keys, and values all derived from one sequence. The scaled dot product, with the $1/\sqrt{d_k}$ factor derived from the variance of an inner product of d_k independent unit-variance components. Multi-head attention as multiple representation subspaces at reduced dimension. Complexity analysis: $O(n^2 d)$ per layer against $O(nd^2)$ for recurrence, and the crossover at $n \approx 6d$. **Maximum path length between dependent positions — $O(1)$ for self-attention, $O(n)$ for recurrence, $O(\log_k n)$ for dilated convolution.** Table 1 of Vaswani et al. is the argument for the entire architecture and should be reconstructed on the board.

Objectives: derive the $\sqrt{d_k}$ scaling; compute attention FLOPs for a given sequence length and model dimension; explain what multi-head buys over single-head at equal parameter count.

Readings: Bahdanau et al. (2015); Luong et al. (2015); Xu et al. (2015) §3–4; Vaswani et al. (2017) §1–3; `technical-notes.pdf` §10–11. **Lab 6: Attention, and the payoff** — implement it, then bolt it onto your own Lab 5 translator and re-plot the curve. **PS2 due. PS3 out.**

Week 7 · The Transformer

L1 — The architecture in full. Encoder and decoder stacks. Causal masking and its implementation. Position-wise feed-forward networks and the 4× expansion ratio. Residual connections and layer normalization — Weeks 3 and 4 paying off. Post-LN versus pre-LN placement, and why the original post-LN formulation requires warmup while pre-LN does not (Xiong et al., 2020). Sinusoidal positional encoding, derived from the requirement that relative offsets be expressible as linear transformations, plus the deeper point that permutation equivariance makes *some* position signal mandatory. The training recipe: Adam with inverse-square-root warmup, label smoothing, dropout placement. Parameter and FLOP accounting: $\approx 12Ld^2$ parameters, $\approx 6N$ FLOPs per token of training.

Then read Table 3 line by line. Which ablation carries the claim? What happens when you vary head count at fixed total dimension? What does the learned-versus-sinusoidal positional-encoding row actually show?

L2 — The unification. Following Isola: MLPs, CNNs, GNNs, and Transformers as one family. Attention is a data-dependent mixing matrix over tokens; convolution is the same operation with a fixed, local, shared kernel; the MLP is the degenerate case with no mixing; a GNN is attention restricted to a given adjacency. This reframing is what lets the Transformer be applied to images (ViT), point clouds, and graphs without reinvention. Dosovitskiy et al. as the demonstration that the grid prior was never necessary given enough data.

Objectives: implement causal masking correctly; compute the parameter count of a specified Transformer; justify pre-LN versus post-LN; express convolution as constrained attention.

Readings: **Vaswani et al. (2017) in full, twice.** Rush, *The Annotated Transformer*; Alammar, *The Illustrated Transformer*; Xiong et al. (2020); Dosovitskiy et al. (2021); `technical-notes.pdf`

§12–14. **Project proposals due.**

PART III — WHY ANY OF IT WORKS

Week 8 · Generalization

L1 — The failure of classical capacity control. Zhang et al. (2017): networks that fit random labels perfectly still generalize on real labels, which falsifies any purely capacity-based explanation. The inadequacy of VC dimension and Rademacher bounds at these parameter counts. Overparameterization. Double descent (Belkin et al., 2019; Nakkiran et al., 2020) in model size, data size, and training time. Implicit regularization of gradient descent — the minimum-norm interpolation result for linear models as the tractable case. The neural tangent kernel and the NN–Gaussian-process correspondence as the two regimes where analysis is possible, plus honest discussion of why the infinite-width limit misses feature learning.

Assign both Zhang et al. and Wilson's *Deep Learning Is Not So Mysterious or Different*. **The disagreement is the lecture.**

Objectives: explain why random-label memorization refutes capacity-based accounts; describe the double-descent curve and identify the interpolation threshold; state what NTK explains and what it cannot.

L2 — Midterm. Two hours, closed book, one double-sided handwritten sheet permitted. Scope is Weeks 1–7 and technical notes §1–§14, i.e. **items 1–23 and 25** of the exam-eligible list in §22. Item 24, the RoPE identity, sits in §13.3 but is Week 12 material and is not on this paper; §15–§19 and items 26–34 are out, and Week 1's optimisation material is examinable as *statements* rather than derivations. Laboratory results are examinable as *results* — you may be asked what a measurement showed and why — but no laboratory code is examinable.

Readings: Zhang et al. (2017); Belkin et al. (2019); Nakkiran et al. (2020); Jacot et al. (2018); Lee et al. (2018); Wilson (2025). **Lab 8: Memorization** — deliberately the lightest of the term at the desk; its runs go to the queue.

Week 9 · Optimization at scale

L1 — The spectral perspective. Steepest descent is defined relative to a norm; SGD is steepest descent in the Euclidean norm, which is the wrong norm for a layered architecture. Operator-norm and spectral views of weight updates. Why learning rates fail to transfer across width under standard parameterization, and how maximal-update parameterization (μP) fixes it — the scaling table for initialization variance and per-layer learning rate as a function of fan-in. Hyperparameter transfer: tune at small width, deploy at large. Feature learning versus the lazy/NTK regime as the thing μP is designed to preserve. Recent optimizers in this lineage: Shampoo, Muon, modular-norm methods.

L2 — Distributed training and batch scaling. Gradient noise scale and critical batch size. The linear scaling rule and warmup. Data, tensor, pipeline, and sequence parallelism.

ZeRO/FSDP sharding of optimizer state, gradients, and parameters. Mixed precision, loss scaling, and where numerical issues actually bite. FlashAttention as an IO-aware reformulation — the point is that the algorithm did not change, the memory hierarchy usage did.

Objectives: state how learning rate must scale with width under μP; compute the memory footprint of Adam states for a given model; explain what FlashAttention changes and what it does not.

Readings: Yang & Hu (2021); Yang et al. (2022); McCandlish et al. (2018); Goyal et al. (2017); Rajbhandari et al. (2020); Dao et al. (2022); `technical-notes.pdf` §15. **Lab 9: Coordinate check and roofline** — reserved cluster window, runs overnight. **PS3 due. PS4 out.**

Week 10 · Representation learning

L1 — Reconstruction-based. What a representation is and what makes one good. Autoencoders, bottlenecks, denoising. Vector quantization. Masked prediction as a pretext task: BERT's masked language modeling and MAE's masked image modeling as the same idea in two modalities. Why reconstruction objectives can waste capacity on perceptually irrelevant detail.

L2 — Similarity-based. Metric learning. Contrastive objectives and InfoNCE as a lower bound on mutual information. Hard negatives and why they matter. Alignment and uniformity as the two properties a contrastive loss optimizes. SimCLR, MoCo, BYOL — and the puzzle of why BYOL works without negatives. DINO. CLIP as cross-modal contrastive learning and the source of most current multimodal capability.

Objectives: derive InfoNCE as a mutual-information bound; explain the role of the temperature parameter; describe the collapse failure mode and the mechanisms that prevent it.

Readings: Bengio et al. (2013); van den Oord et al. (2018); Chen et al. (2020); He et al. (2020, 2022); Grill et al. (2020); Caron et al. (2021); Radford et al. (2021); Wang & Isola (2020).

PART IV — MODERN PRACTICE

Week 11 · Pretraining and scaling

L1 — Pretraining. BERT and GPT as encoder-only and decoder-only specializations of Week 7 — the same block, different masking and objective. T5 and the text-to-text framing. What pretraining data actually is: Common Crawl processing, quality filtering, deduplication, contamination. Tokenization at scale and its downstream consequences, including the cost asymmetry across languages.

L2 — Scaling laws. Kaplan et al. (2020): power-law relationships between loss, parameters, data, and compute. Hoffmann et al. (2022) and the correction — compute-optimal training scales parameters and data roughly equally, so most large models of the era were badly undertrained. The IsoFLOP methodology. Then the emergence dispute: Wei et al. (2022) report discontinuous capability jumps; Schaeffer et al. (2023) argue these are artifacts of discontinuous metrics. **Teach this as an open disagreement about measurement, not a settled result.**

Objectives: derive the Chinchilla-optimal allocation from a compute budget; explain why $C \approx 6ND$; articulate both sides of the emergence dispute.

Readings: Kaplan et al. (2020); Hoffmann et al. (2022); Wei et al. (2022); Schaeffer et al. (2023); Devlin et al. (2019); Brown et al. (2020); `technical-notes.pdf` §16. **Lab 11: Scaling laws on a budget** — a class-pooled experiment on three iso-FLOP contours; reserved overnight window. **PS4 due. PS5 out.**

Week 12 · Post-training and adaptation

L1 — Alignment. Supervised fine-tuning and instruction tuning. Reinforcement learning from human feedback: the reward model, PPO, and the practical difficulties. Direct preference optimization and the derivation showing the language model is implicitly its own reward model. Where RLHF changes capability versus where it changes only style.

L2 — Efficient adaptation and long context. Parameter-efficient methods: adapters, LoRA and its low-rank hypothesis, prefix tuning. In-context learning and the induction-head account of where it comes from. Chain-of-thought prompting. Positional-encoding successors — relative position representations, RoPE and its rotation-invariance derivation, ALiBi — and context-length extension.

Objectives: derive the DPO objective from the RLHF one; state LoRA's parameter savings for a given rank; explain why RoPE's inner product depends only on relative position.

Readings: Ouyang et al. (2022); Rafailov et al. (2023); Hu et al. (2021); Shaw et al. (2018); Su et al. (2021); Press et al. (2022); Olsson et al. (2022); Wei et al. (2022b).

Week 13 · The practitioner's week

L1 — Getting networks to work. Both MIT and Stanford dedicate a lecture to this and it is not filler. Overfit a single batch before anything else. The bisection discipline for locating a bug. Learning-rate range tests. Reading a loss curve: what a plateau, a spike, a sawtooth, and a slow divergence each indicate. Baselines that must be beaten before a result means anything. Seeds, variance across runs, and the difference between an improvement and noise. Common silent failures: label leakage, train/test contamination, a validation set that drifted.

L2 — Evaluation. Designing a benchmark. Metric selection and the ways metrics mislead — BLEU's decade of distortion in machine translation is the closing of Week 5's loop. Contamination detection. MMLU and HELM as contrasting designs. LLM-as-judge, position bias, and self-preference. Human evaluation and its cost structure.

Objectives: design an evaluation for a stated capability and name its failure modes; identify contamination in a described setup; justify a baseline choice.

Readings: Karpathy, *A Recipe for Training Neural Networks*; Zinkevich, *Rules of Machine Learning*; Goodfellow et al. Ch. 11; Papineni et al. (2002); Hendrycks et al. (2021); Liang et al. (2022); Zheng et al. (2023). **Lab 13: The metric is an instrument. PS5 due.**

Week 14 · Inference, retrospect, open questions

L1 — Inference-time computation. Everything past a single forward pass. Beam search and sampling strategies. Chain-of-thought, self-consistency, and best-of-n. Test-time compute scaling and the tradeoff against training compute. Speculative decoding. Test-time training.

L2 — Retrospect. Was recurrence genuinely unnecessary, or did attention win because it parallelizes on hardware that already existed? Hooker's *hardware lottery* as the frame. The quadratic-cost problem and the efficient-attention literature. State-space models and Mamba as recurrence's return with a parallel scan — the sequential/parallel tradeoff reopened rather than closed. Course synthesis: every component of the Transformer traces to a failure this course examined.

Readings: Snell et al. (2024); Wang et al. (2023); Hooker (2021); Gu & Dao (2023); Tay et al. (2022). **Final projects due. Poster session.**

---

6. Problem sets

Five sets, **5% each**, on the schedule in §5. Each is Part A (derivations, 35%) and Part B (implementation, 65%), and the two are linked — at least one Part A result is checked numerically by Part B code. **For PS1 and PS2 you may not call a library implementation of anything the set asks you to write**; building a Transformer with `torch.nn.MultiheadAttention` teaches nothing. From PS3 onward the framework is assumed, because the object of study has moved from *how is a gradient computed* to *what happens at scale*.

These sets are shorter than in earlier editions, and that is not a reduction in coverage. The mechanical, hands-on layer they used to open with is now the laboratory track, assessed weekly rather than fortnightly. What remains is the hard core. Full specifications are in `assignments.pdf`.

PS1 — An engine that differentiates *(out Wk 1, due Wk 3)*

Part A. (E) The affine VJP by indices and by shapes. (E) The softmax Jacobian and the fused softmax–cross-entropy collapse to $\bar{z} = s - y$, with both independent reasons for the fusion and an input at which the unfused form loses digits. (E) The layer recursion and its submultiplicative bound, with one sentence per response in the Week 2 table naming which factor it attacks. The $\sqrt{k}$ checkpointing optimum. (E) $\varepsilon^\star \sim \delta^{1/3}$ for the central difference, and the resulting best relative error in float32 and float64.

Part B. A reverse-mode autodiff engine over NumPy arrays: tape, topological sort, VJP registration. Correct *accumulation* for nodes used more than once, and correct broadcasting in the backward pass. A VJP catalogue (`matmul`, `add`, `mul`, `div`, `relu`, `tanh`, `exp`, `log`, `sum`, `mean`, `reshape`, `transpose`, `softmax`, fused `cross_entropy`). A gradient-checking harness implementing the protocol from lecture, with ε chosen by your Part A result. Checkpointing as a wrapper, with bit-identical outputs. Train a 3-layer MLP on Fashion-MNIST to $\geq 88\%$. **Then: the**

starter repository contains one VJP that is subtly wrong. Find it with your own harness and say what class of input exposes it.

PS2 — Convolution, residuals, normalization *(out Wk 4, due Wk 6)*

Part A. (E) Receptive-field arithmetic for a stack of (kernel, stride, dilation) triples, applied to a supplied configuration and then to the same one with a stride moved, taking the field $9 \to 21$ at unchanged parameter count. (E) The BatchNorm backward pass by the three-path decomposition, shown equivalent to projecting out two named directions. (E) The residual Jacobian $I + \partial F/\partial x$, the 2^L path expansion, and precisely what the ensemble reading does and does not claim. (E) Scale invariance and the $\eta/\|W\|^2$ effective rate. Santurkar against Ioffe & Szegedy — and the actual question: what would have had to be true for the original explanation to be correct?

Part B. NumPy `conv2d` forward and backward via im2col/col2im with stride, padding and groups, starting from your Lab 4 forward path. NumPy BatchNorm forward and backward from your own Part A derivation, plus the `layer_norm` VJP. **Part A and Part B must agree; if they do not, say so.** Then in PyTorch: ResNet-20 and ResNet-56 with residual and normalization switches, and the ablation grid — three seeds at depth 20, one at depth 56. Reach $\geq 89.5\%$ on CIFAR-10 in 30 epochs and report the gap to the published 180-epoch result. Measure $\eta/\|W\|^2$ over training against the nominal η.

PS3 — Attention and the Transformer *(out Wk 6, due Wk 9)*

Part A. (E) $\mathrm{Var}[q^\top k] = d_k$ and hence the scaling, with the logit standard deviation at $d_k = 64$ without it, connected to the Week 3 saturation failure it reproduces. (E) The backward pass of scaled dot-product attention in matrix form. Multi-head parameter invariance, and why an interior optimum in h exists anyway — an argument about what is representable, not about parameters. (E) The FLOP crossover at $n > 6d$, and separately why the $O(n^2)$ memory cost binds where the FLOP cost does not. (E) Sinusoidal encoding from the shift requirement, then Table 3 row (E) against the motivation you just derived.

Part B. A complete Transformer from tensor primitives — no `nn.MultiheadAttention`, no `nn.Transformer`. Pre-LN and post-LN as a switch, learned and sinusoidal encodings as a switch, causal and padding masks, greedy and beam decoding. Match a supplied reference forward pass to 10^{-4}, and verify causality by asserting invariance to changes at later positions. Train on Multi30k de→en. **Reproduce one row of Table 3**, assigned per student and pooled across the cohort, capped at four configurations and three seeds — and report whether your effect size exceeds seed variance. For at least one row it will not, and saying so earns more than a large number. Implement both FLOP functions, with and without the $4n^2d$ term, and mark the crossing.

PS4 — Scale *(out Wk 9, due Wk 11)*

Part A. (E) $\Delta z = \Theta(\eta n)$ for an Adam update, explicit about where coherence rather than cancellation is assumed. (E) The μP exponents for Adam, then the SGD exponents and one sentence on why they differ. Standard parameterization's infinite-width limit as the lazy/NTK limit. Gradient noise scale and critical batch size. The memory footprint of training a 7B model

in mixed precision with Adam, recomputed under ZeRO stages 1–3 at 8-way sharding — and what mixed precision does *not* reduce.

Part B. Extend your Lab 9 μP wrapper to a model with attention; the coordinate check must pass from width 128 to 4096. Demonstrate transfer by tuning at width 256 and comparing against a *supplied* width-4096 sweep. Run DDP and FSDP/ZeRO-3 on four GPUs and explain the scaling gap. Compare fp32, bf16 and fp16-with-loss-scaling, and name one operation you had to keep in fp32. Profile a training step and attribute time by component. Measure standard against fused attention at sequence lengths 512–8192 — **and verify that the FLOP count is identical across the two.**

PS5 — Pretrain, adapt, evaluate *(out Wk 11, due Wk 13)*

Part A. (E) The Chinchilla allocation from $C = 6ND$, obtaining $N_{\text{opt}} \propto C^{\beta/(\alpha+\beta)} \approx C^{0.45}$ — and the observation that a scale-*independent* tokens-per-parameter ratio would require the exponent to be exactly 0.50, so "20 tokens per parameter" is a value at a scale, not a constant. The Kaplan cosine-schedule artifact: what was held fixed, and why the bias was systematic rather than noise. (E) DPO in three steps, with two honest caveats. Bradley–Terry identifiability, and why it is exactly what makes step three work. The emergence dispute, and the question that decides it — which is a question about the metric.

Part B. Pretrain a 30M decoder-only LM with your own trained BPE tokenizer; report the vocabulary size you chose and the compression ratio in two languages. **Then, before evaluating anything, audit for contamination** by n-gram overlap, decontaminate, and state what your method would miss. From here on you work with a supplied 1B base model, because a 30M model has no instruction-following behaviour to align: SFT it, implement DPO from scratch and align it, sweep β and plot the reward–KL frontier. Design a 60-item benchmark with a stated construct and a contamination-resistance argument, evaluate all three models with binomial confidence intervals, then evaluate them again with an LLM judge under position-swap and length control. Where the two rankings disagree, say which you believe and why.

---

7. Final project

30% of the grade. Groups of up to three. Larger teams are expected to attempt correspondingly more.

Format: a research blog post

Not a conference paper. A written investigation with figures, plots, and where useful animations or interactive elements — the Distill house style. Graded on **clarity and insight as heavily as on novelty**. A well-explained negative result that isolates why something fails will score above an unexplained improvement.

The post must contain: background sufficient for a classmate to follow, a clearly stated question, the experiments run, the results with honest error bars, and a discussion of what the reader should now believe that they did not before.

Two routes

Default project. Implement a minimal GPT-2 and evaluate it on three downstream tasks. For students who would rather spend their effort on implementation than on scoping. No mentor assigned.

Custom project. Your own question. A staff mentor is assigned after proposals. Acceptable shapes:

- *Reproduction.* Reproduce a syllabus result at reduced scale and report where it fails to hold.
- *Ablation.* Take a component we studied and test whether the original justification survives scrutiny.
- *Application.* Apply a studied architecture to a domain you know — irregularly sampled series, structured tabular data, scientific measurements.
- *Theory.* A careful exposition or small extension: depth separation, signal propagation, a scaling-rule derivation.

Deadlines

Milestone	Week	Weight
Proposal (2 pages)	7	5%
Milestone report	11	5%
Final post	14	17%
Poster session	14	3%

---

8. Notes on design

Four choices differ from common practice and are worth stating.

The Transformer arrives at the midpoint, not the end. Every comparable course front-loads it further still — Week 3 of 10 at CS224n, Week 4 at MIT 6.7960 and CS231n, Assignment 1 at CS336. The structural reason is that projects cannot use an architecture the lectures have not reached. This course keeps the motivating arc — bottleneck measured in Week 5, attention derived in Week 6, architecture assembled in Week 7 — and then spends the entire second half on what the paper does not explain. Both goals are achievable in a semester; only one is achievable in a quarter.

Problems precede solutions, empirically. Lab 5 exists so that students *measure* BLEU degrading with sentence length before anyone says "attention." The Transformer is far less mysterious when it answers a question the student already has.

Approximation theory and μP are non-negotiable. They are the highest-value content most curricula omit. Without the first, "universal approximation" gets cited as though it explained something. Without the second, hyperparameter tuning remains folklore.

Every board result is measured. This is what the laboratory track is, and it is why the problem sets could be shortened to make room for it. The design constraint is that knowing in advance what will be measured must not help you — and it does not, because the answer is a number from your own run. That property is why almost nothing in this assessment needs rotating between cohorts. Three things do: Lab 2's six planted bugs, PS1's planted wrong VJP, and the Table 3 row assignment in PS3.